Smart Data
WORLD CLASS EDUCATION

Sesión 04 – App Engine, Cloud Run Function, Cloud Run

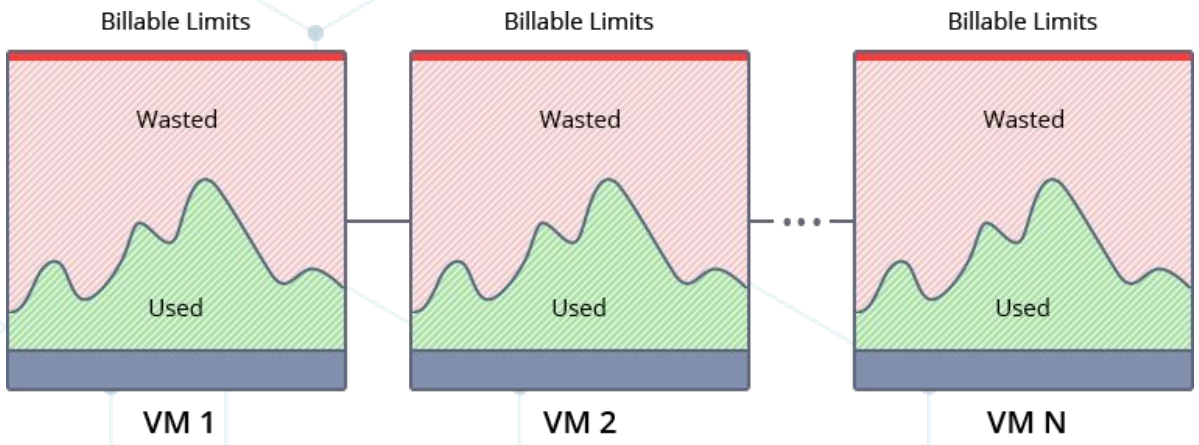
Agenda

1. Introducción a las aplicaciones sin servidor en la nube.
2. App Engine.
3. Cloud Run Functions.
4. Cloud Run.
5. Comentarios finales.

1. Introducción a las aplicaciones sin servidor en la nube

¿Qué es la computación sin servidor?

- Modelo de ejecución en al nube donde el proveedor gestiona la infraestructura.
- Los desarrolladores solo se preocupan por el código.
- Pago basado en el uso real de recursos.



Definición y Ventajas

Definición:

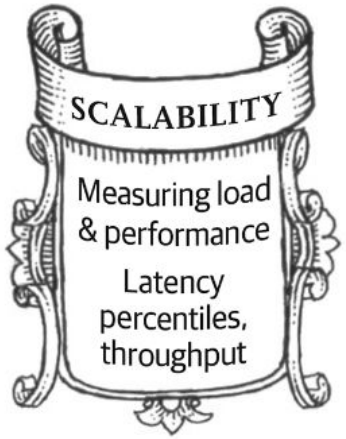
- No es necesario gestionar servidores.
- Despliegue de código directo a la nube.

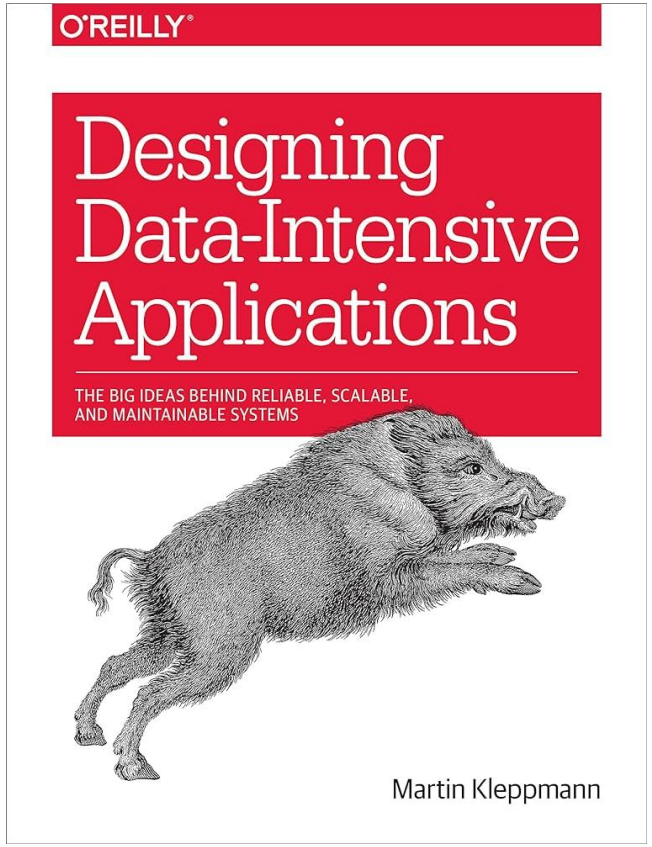
Ventajas:

- **Escalabilidad Automática:** Ajuste automático a la demanda.
- **Costos:** Pago por uso, sin costos fijos.
- **Desarrollo Rápido:** Enfoque en la lógica de negocio.
- **Mantenimiento Sencillo:** Sin gestión de infraestructura.

Para Quienes Desean Ir Más Allá

- Escalabilidad.
- Consistencia.
- Diseño de sistemas distribuidos.





Casos de Uso Comunes

- **Aplicaciones Web y Móviles**: Backend para aplicaciones ligeras y escalables.
- **Procesamiento de Datos**: ETL, procesamiento de flujos de datos en tiempo real.
- **Automatización de Tareas**: Scripts y cron jobs.
- **Microservicios**: Despliegue y gestión de servicios pequeños y autónomos.

2. App Engine

- Principal herramienta de GCP en lo que refiere a "managed services".
- Lanzado en 2008.
- Serverless.
- PaaS.

Ventajas de utilizar App Engine

- Es la forma más simple de desplegar y escalar aplicaciones en GCP.
- Soporte para múltiples lenguajes de programación.

Go	Java	Node.js	PHP
Python	Ruby		

- Características:
  - Pay as you go.
  - Load balancing y escalamiento automático.
  - Versionamiento de aplicaciones.
  - Traffic splitting.

Compute Engine vs App Engine

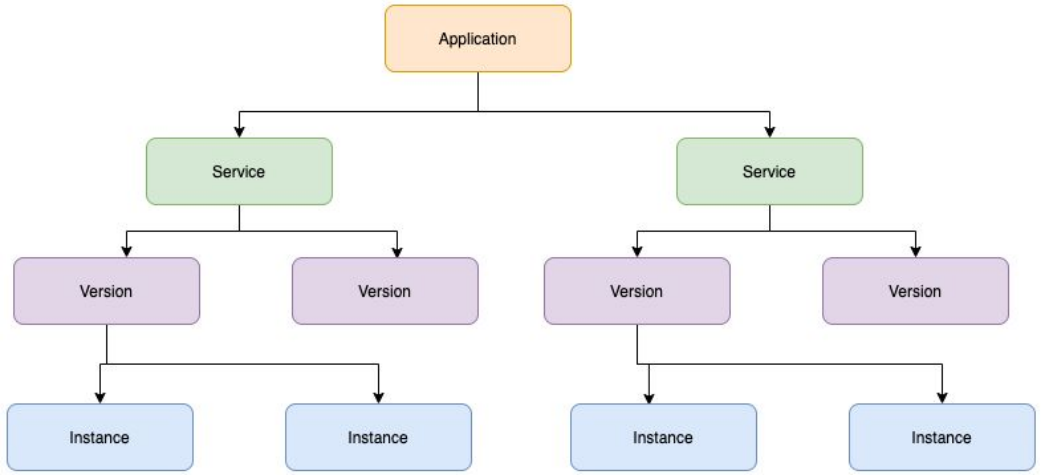
- **Compute Engine**
  - IaaS
  - MAYOR Flexibilidad
  - MAYOR Responsabilidad
    - Elegir imagen (SO)
    - Instalar Software
    - Elegir Hardware
    - Accesos (Certificados/Firewalls)
    - Disponibilidad/Escalabilidad
- App Engine
  - PaaS
  - MENOR Flexibilidad
  - MENOR Responsabilidad

App Engine environments

- **Standard:** Aplicaciones ejecutándose en Sandboxes con un lenguaje de programación específico.
  - V1: Java, Python, PHP, Go (versiones antiguas).
    - Python y PHP
      - Acceso restringido.
      - Lista blanca de librerías para ser instaladas.
    - Java y Go sin restricciones
  - V2: Java, Python, PHP, Node.js, Ruby, Go (versiones recientes).
    - Sin restricciones de librerías o accesos.
  - Puede escalar a cero.
- **Flexible:** Aplicaciones en Docker containers.
  - Utiliza Compute Engine VMs.
  - Soporte para cualquier lenguaje siempre que esté en un container.
  - Puede conectarse con discos locales.
  - Como mínimo, puede escalar a una instancia.

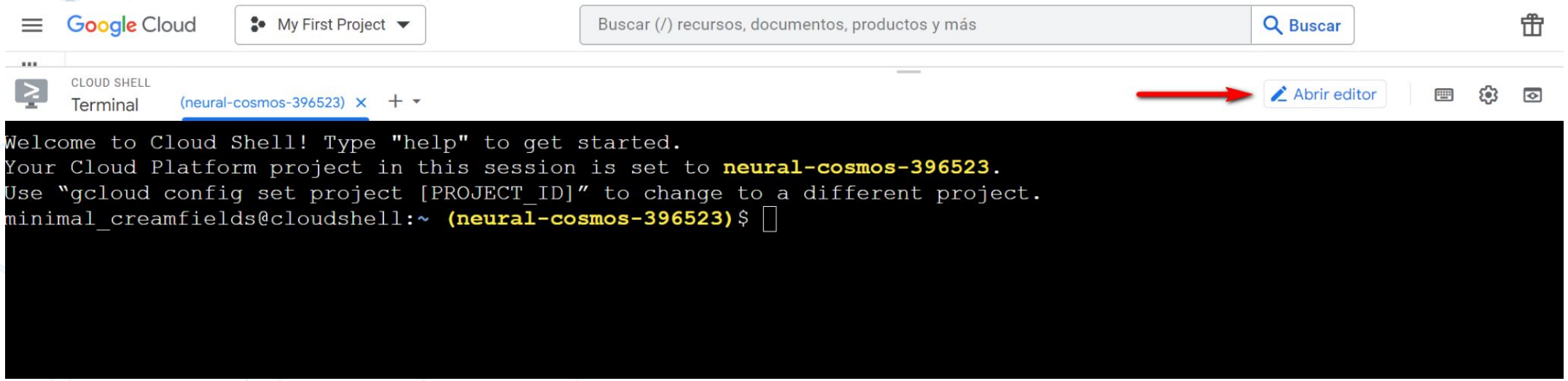
Jerarquía de Componentes

- **Aplicación:** Una por proyecto.
- **Servicio(s):** Varios microservicios o componentes.
  - Cada servicio puede tener diferente configuración.
- **Versión:** Cada versión refiere a código y configuración
  - Cada versión puede correr en una o más instancias.
  - Diferentes versiones pueden co-existir.
  - Opciones para rollback y distribución de tráfico.



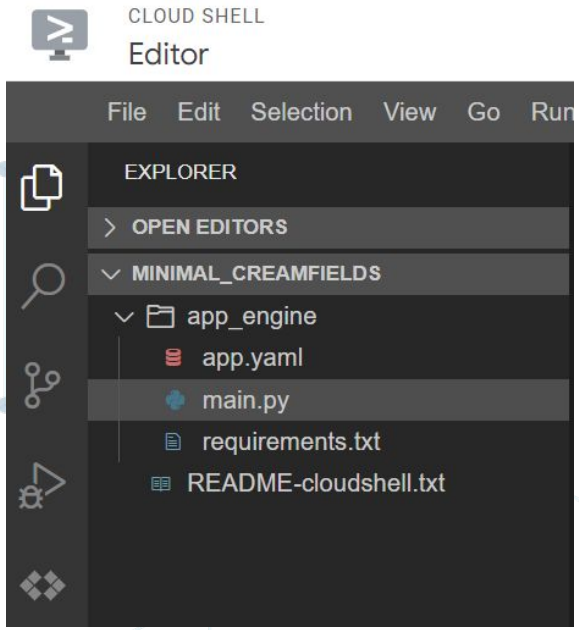
Laboratorio de App Engine

1. Activamos Cloud Shell y abrimos editor



Laboratorio 01:

- **Cloud Shell:**
  - Creamos una carpeta app_engine con los siguientes archivos:
    - app.yaml
    - main.py
    - requirements.txt



Laboratorio 01:

- app.yaml

```
runtime: python39
```

For other configurations, see:

- Env: flexible - https://cloud.google.com/appengine/docs/flexible/python/configuring-your-app-with-app-yaml
- Autoscaling - https://cloud.google.com/appengine/docs/legacy/standard/python/config/appref#scaling_elements
- Traffic spliting: https://cloud.google.com/appengine/docs/flexible/splitting-traffic

Laboratorio 01:

- main.py

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return 'Mi primera API con flask'

if __name__ == '__main__':
    app.run()
```

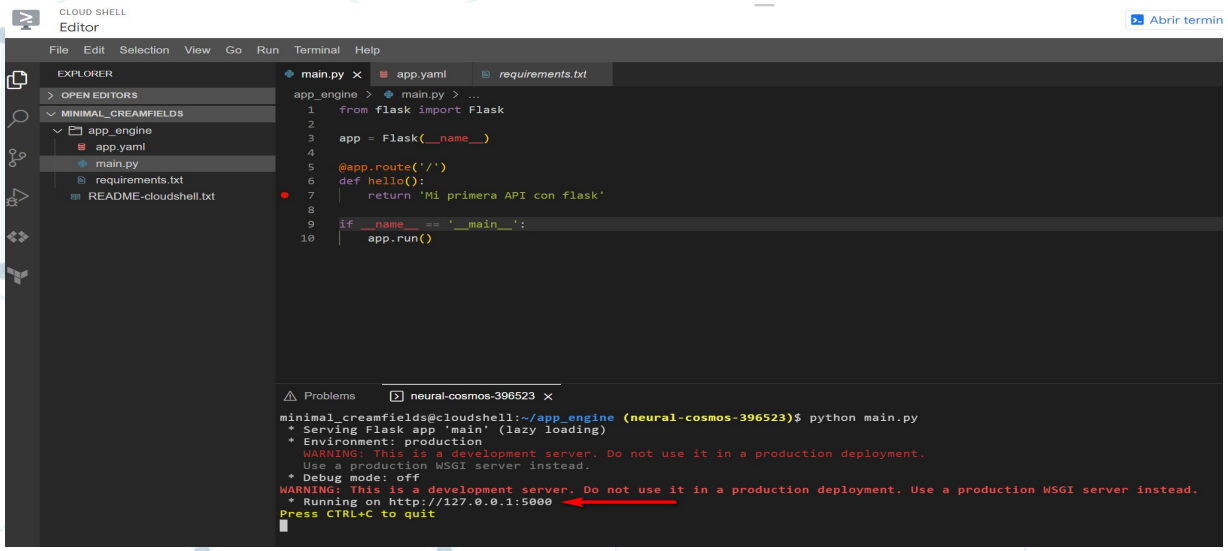
Laboratorio 01:

- requirements.txt

```
Flask==3.0.0
```

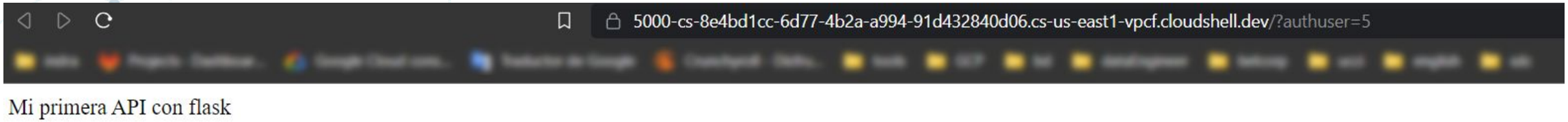
Laboratorio 01:

- **Cloud Shell:**
  - Ejecutamos en terminal del editor:
    - python -m venv venv_app_engine
    - source venv_app_engine/bin/activate
    - pip install -r requirements.txt
    - python main.py



Laboratorio 01:

- Validamos endpoint,



- Luego ejecutamos ctrl + c en terminal

Laboratorio 01:

- **Terminal de Editor:**
  - Ejecutamos:
    - gcloud app deploy
    - Ingresamos “y”

```
Problems        neural-cosmos-396523 ×

minimal_creamfields@cloudshell:~/app_engine (neural-cosmos-396523)$ gcloud app deploy
Services to deploy:

descriptor:                  [/home/minimal_creamfields/app_engine/app.yaml]
source:                      [/home/minimal_creamfields/app_engine]
target project:              [neural-cosmos-396523]
target service:              [default]
target version:              [20230901t211041]
target url:                  [https://neural-cosmos-396523.uc.r.appspot.com]
target service account:      [neural-cosmos-396523@appspot.gserviceaccount.com]


Do you want to continue (Y/n)?
```

Laboratorio 01:

- **Terminal de Editor:**
  - Validamos servicios creados en app engine:
    - gcloud app services list

```
Problems        neural-cosmos-396523 ×

minimal_creamfields@cloudshell:~/app_engine (neural-cosmos-396523)$ gcloud app services list
SERVICE: default
NUM_VERSIONS: 1
minimal_creamfields@cloudshell:~/app_engine (neural-cosmos-396523)$
```

Laboratorio 01:

- **Terminal de Editor:**
  - Validamos versiones creadas:
    - gcloud app versions list

```
Problems        neural-cosmos-396523 ×

minimal_creamfields@cloudshell:~/app_engine (neural-cosmos-396523)$ gcloud app versions list
SERVICE: default
VERSION.ID: 20230901t211041
TRAFFIC_SPLIT: 1.00
LAST_DEPLOYED: 2023-09-01T21:12:06+00:00
SERVING_STATUS: SERVING
minimal_creamfields@cloudshell:~/app_engine (neural-cosmos-396523)$
```

Laboratorio 01:

- **Terminal de Editor:**
  - Validamos endpoint de nuestro app engine:
    - gcloud app browse

```
Problems        neural-cosmos-396523 ×

minimal_creamfields@cloudshell:~/app_engine (neural-cosmos-396523)$ gcloud app browse
Did not detect your browser. Go to this link to view your app:
https://neural-cosmos-396523.uc.r.appspot.com
minimal_creamfields@cloudshell:~/app_engine (neural-cosmos-396523)$
```

Laboratorio 01:

- **Terminal de Editor:**
  - Actualizamos nuestro main.py
  - Ejecutamos
    - gcloud app deploy

main.py × app.yaml requirements.txt

app_engine > main.py > ...

```python
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return 'Mi primera API con flask version 2'

if __name__ == '__main__':
    app.run()
```

```
minimal_creamfields@cloudshell:~/app_engine (neural-cosmos-396523)$ gcloud app deploy
Services to deploy:

descriptor:                  [/home/minimal_creamfields/app_engine/app.yaml]
source:                      [/home/minimal_creamfields/app_engine]
target project:              [neural-cosmos-396523]
target service:              [default]
target version:              [20230901t212206]
target url:                  [https://neural-cosmos-396523.uc.r.appspot.com]
target service account:      [neural-cosmos-396523@appspot.gserviceaccount.com]


Do you want to continue (Y/n)?
```

Laboratorio 01:

- **Terminal de Editor:**
  - Validamos versiones creadas:
    - gcloud app versions list

```
Problems        neural-cosmos-396523 ×

minimal_creamfields@cloudshell:~/app_engine (neural-cosmos-396523)$ gcloud app versions list
SERVICE: default
VERSION.ID: 20230901t211041
TRAFFIC_SPLIT: 0.50
LAST_DEPLOYED: 2023-09-01T21:12:06+00:00
SERVING_STATUS: SERVING

SERVICE: default
VERSION.ID: 20230901t212206
TRAFFIC_SPLIT: 0.50
LAST_DEPLOYED: 2023-09-01T21:23:22+00:00
SERVING_STATUS: SERVING
minimal_creamfields@cloudshell:~/app_engine (neural-cosmos-396523)$
```

Laboratorio 01:

- **Terminal de Editor:**
  - Validamos versiones creadas:
    - gcloud app browse --version 20231101t010304
    - gcloud app browse --version 20231101t005436
  - Distribuimos el tráfico:
    - gcloud app services set-traffic --splits=20231101t005318=.5,20231101t004754=.5
    - gcloud app services set-traffic --splits=20231101t010304=.5,20231101t005436=.5 --split-by=random
  - Evaluamos el endpoint
    - gcloud app browse
    - watch curl https://balmy-moonlight-396917.uc.r.appspot.com

3. Cloud Run Functions

- **Descripción**:
  - Servicio de computación sin servidor para ejecutar funciones en respuesta a eventos.

- **Características**:
  - Respuesta a eventos de Google Cloud Storage, HTTP, y otros.
  - Escalabilidad automática.
  - Pago por invocación y tiempo de ejecución.

Cloud Run Functions

Tipos de Cloud Run Functions:

- **HTTP Functions:** Estas funciones se activan mediante solicitudes HTTP. Son ideales para crear API RESTful, microservicios y aplicaciones web que responden a peticiones HTTP.

- **Background Functions:** Se activan por eventos en otros servicios de GCP, como Cloud Storage, Cloud Pub/Sub y Firestore. Pueden procesar eventos como carga de archivos, mensajes en cola o cambios en bases de datos.

Cloud Run Functions

Eventos y Disparadores(Triggers):

- Los eventos son acciones o cambios que ocurren en GCP, como la creación de un archivo en Cloud Storage o la llegada de un mensaje a Pub/Sub. Los disparadores son configuraciones que definen cuándo se debe ejecutar una Cloud Function en respuesta a un evento específico.

- Imagina una aplicación que sube imágenes a Cloud Storage. Puedes configurar una Cloud Function para redimensionar automáticamente cada imagen cuando se cargue en Cloud Storage. En este caso, el evento es la carga de la imagen y el disparador se configura para activar la función de redimensionamiento.

Cloud Run Functions

Ventajas

- **Sin servidor:** No es necesario administrar ni aprovisionar servidores. Google se encarga de la infraestructura subyacente.
- **Escalabilidad automática:** Las funciones se escalan automáticamente en función de la carga de trabajo, lo que garantiza que tu aplicación pueda manejar picos de tráfico.
- **Pago por uso:** Solo pagas por el tiempo de ejecución de sus funciones, lo que puede ser más económico que mantener servidores en funcionamiento constantemente.
- **Amplia integración:** Se integra de manera nativa con otros servicios de GCP, lo que facilita la creación de aplicaciones completas.

Cloud Run Functions

Datos importante y más relevantes

- Cloud Run Functions se escala automáticamente desde cero hasta miles de instancias de función en respuesta a la carga, lo que lo hace adecuado para aplicaciones con picos de tráfico impredecibles.
- Puedes configurar desencadenadores y filtros para controlar cuándo se activan tus funciones.
- Cloud Run Functions ofrece un plan gratuito con un límite de invocaciones mensuales, lo que te permite comenzar a usar el servicio sin costo inicial.

Cloud Run Functions

Casos de uso

Cloud Run Functions es adecuado para una amplia gama de casos de uso, entre los que se incluyen:

- **Tareas automatizadas:** Cloud Functions se puede utilizar para automatizar tareas, como la actualización de datos, el envío de correos electrónicos o la notificación de eventos.
- **APIs:** Cloud Functions se puede utilizar para crear APIs que se utilizan por otras aplicaciones.
- **Procesamiento de datos:** Cloud Functions se puede utilizar para procesar datos, como la analítica de datos o el aprendizaje automático.
- **Web scraping:** Cloud Functions se puede utilizar para raspar datos de sitios web.

Laboratorio 02:

- Crear una instancia de Cloud Function desde la Consola

Cloud Run

¿Qué es Cloud Run en GCP?

- Cloud Run es un servicio de Google Cloud Platform (GCP) que permite a los desarrolladores implementar contenedores de Docker de forma fácil y rápida en un entorno completamente administrado y sin servidor. Con Cloud Run, puedes ejecutar aplicaciones en contenedores de forma escalable y pagar solo por el tiempo de ejecución de tus contenedores, lo que lo convierte en una opción rentable para aplicaciones web y microservicios.

Cloud Run

Tipos de Cloud Run:

- **Cloud Run (totalmente administrado):** Esta opción es completamente administrada por Google Cloud y ofrece una plataforma sin servidor para ejecutar contenedores. Es ideal para aplicaciones web, APIs y microservicios.

- **Cloud Run para Anthos:** Esta variante está diseñada para ejecutarse en el entorno de Kubernetes de Anthos, lo que proporciona mayor flexibilidad y control para las empresas que utilizan Kubernetes en sus propias instalaciones.

Cloud Run

Ventajas de Cloud Run:

- **Escalabilidad automática:** Las aplicaciones se escalan automáticamente en función de la carga de trabajo, lo que garantiza un rendimiento óptimo incluso durante los picos de tráfico.
- **Pago por uso:** Solo pagas por el tiempo de ejecución de tus contenedores, lo que ahorra costos en comparación con la ejecución de servidores permanentes.
- **Fácil implementación:** La implementación de contenedores en Cloud Run es sencilla y rápida, lo que acelera el desarrollo y la puesta en marcha de aplicaciones.
- **Integración con GCP:** Se integra con otros servicios de Google Cloud, como Cloud Logging, Cloud Monitoring y Cloud Trace, para facilitar la supervisión y la depuración de aplicaciones.

Cloud Run

Características clave:

- **Soporte para contenedores de Docker:** Puedes implementar contenedores de Docker en Cloud Run, lo que proporciona flexibilidad en la elección de tecnologías y lenguajes de programación.

- **Escalabilidad automática:** Cloud Run se encarga de la gestión de instancias y el escalado, lo que simplifica la administración de aplicaciones.

- **Conexión con Cloud Build:** Puedes configurar la integración continua y la implementación (CI/CD) con Cloud Build para automatizar la implementación de tus aplicaciones.

Cloud Run

Datos importantes y relevantes:

- Cloud Run se puede usar en conjunto con servicios de administración de contenedores de Google Cloud, como Google Kubernetes Engine (GKE), para obtener mayor control y flexibilidad.
- Puedes definir las asignaciones de recursos para tus contenedores, lo que te permite controlar la cantidad de CPU y memoria que se asignan a cada instancia de contenedor.

Casos de uso de Cloud Run:

- Implementación de aplicaciones web y microservicios basados en contenedores.
- Ejecución de APIs y servicios RESTful sin servidor.
- Implementación de aplicaciones de procesamiento de eventos en tiempo real.
- Ejecución de tareas de procesamiento por lotes de forma escalable.

Laboratorio 03:

Crear una instancia de Cloud Run

Comparación

Característica	Cloud Run	App Engine	Cloud Run Functions
Tipo de Servicio	Serverless contenedores	PaaS (Standard y Flexible Environment)	Serverless funciones
Despliegue	Contenedores Docker (aunque se ha habilitado una opción para despliegue automático desde código)	Código fuente o contenedores (Flexible)	Código fuente
Auto-scaling	Sí, basado en solicitudes	Sí, basado en solicitudes	Sí, basado en eventos
Estado	Stateless	Stateful (Flexible) & Stateless (Standard)	Stateless
Lenguajes Soportados	Cualquier lenguaje soportado por contenedores	Varios lenguajes soportados oficialmente	JavaScript, Python, Go, etc.
Concurrencia	Múltiples solicitudes por instancia	Una o más según configuración	Una solicitud por ejecución
Casos de Uso	Microservicios, APIs, apps basadas en eventos	Aplicaciones web/móviles, microservicios	Procesamiento de eventos, APIs ligeras
Portabilidad	Alta (basado en contenedores)	Menos portátil (dependiente de GCP)	Menos portátil (dependiente de GCP)
Gestión de Infraestructura	Totalmente gestionado	Totalmente gestionado	Totalmente gestionado

Smart Data
WORLD CLASS EDUCATION

Gracias